

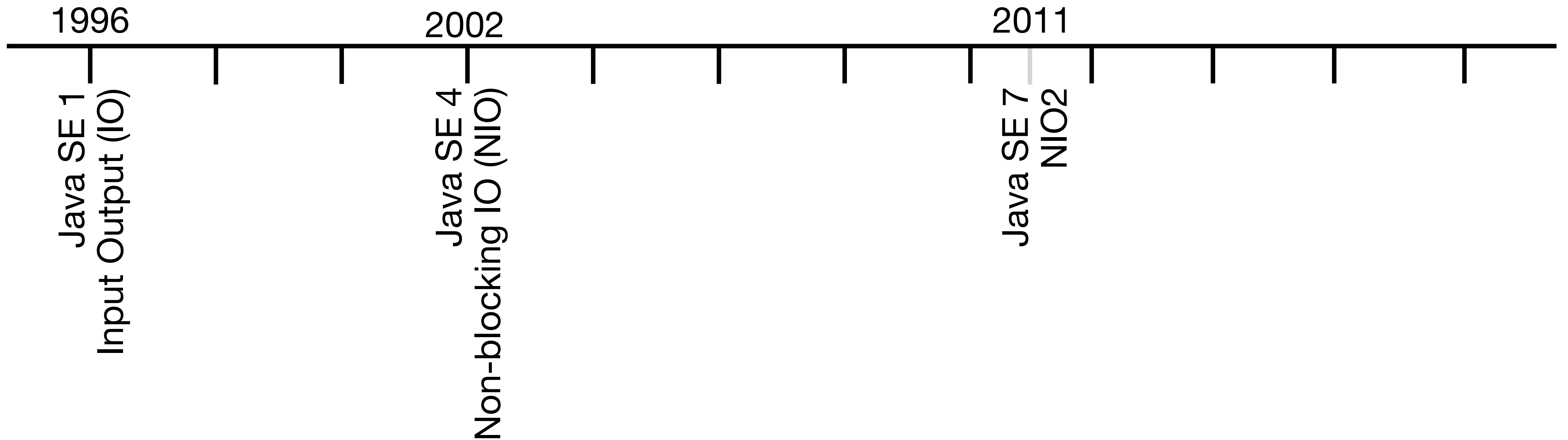
IO & NIO

Java Advanced Developer

StayAhead Training; Oct, 2025

About IO, NIO, & NIO2

IO & NIO



About IO, NIO, & NIO2

IO & NIO

- Classes & interfaces for locating resources & opening streams to them
- `java.io.*` and `java.nio.*`
- `java.io.File` class since Java SE 1 
- `java.nio.file.Path` interface since Java SE 7 

IO Streams

IO & NIO

- An object that represents an input source/output destination
- Source/destination e.g. disk file, device, network socket, process, or array
- Two types of content: characters and bytes
- Two operations: reading and writing
- Not related to `java.util.Streams`

IO Streams: Abstract Base Classes

IO & NIO

	Reading	Writing
Characters	<code>java.io.Reader</code>	<code>java.io.Writer</code>
Bytes	<code>java.io.InputStream</code>	<code>java.io.OutputStream</code>

File: Creation

IO & NIO

```
File file1 = new File("file1.txt");
```

```
File dir1  = new File("dir1");
```

```
File file2 = new File("dir1", "file2.txt");
```

```
File file3 = new File(dir1, "file3.txt");
```

```
File file4 = new File(new URI("file:///home/file4.txt"));
```


File: Getting Elements

IO & NIO

```
File file1 = new File("./file1.txt");
```

```
String name = file1.getName();  
// name ==> file1.txt
```

```
String path = file1.getPath();  
// path ==> ./file1.txt
```

File: Getting Elements

IO & NIO

```
File file1 = new File("./file1.txt");
```

```
String absolutePath = file1.getAbsolutePath();  
// absolutePath ==> /home/./file1.txt
```

```
String canonicalPath = file1.canonicalPath();  
// canonicalPath ==> /home/file1.txt
```


File: Getting Info.

IO & NIO

```
// assuming the file exists on disk  
File file1 = new File("./file1.txt");
```

```
boolean file1IsFile = file1.isFile();  
// file1IsFile ==> true
```

```
boolean file1IsDirectory = file1.isDirectory();  
// file1IsDirectory ==> false
```

File: Getting Info.

IO & NIO

```
// assuming the file exists & is readable  
File file1 = new File("./file1.txt");
```

```
boolean file1Exists = file1.exists();  
// file1Exists ==> true
```

```
boolean file1Readable = file1.canRead();  
// file1Readable ==> true
```

File: Getting Info.

IO & NIO

```
// assuming the file exists & is writable but not executable  
File file1 = new File("./file1.txt");
```

```
boolean file1Writable = file1.canWrite();  
// file1Writable ==> true
```

```
boolean file1Executable = file1.canExecute();  
// file1Executable ==> false
```

```
// see also setReadable, setWritable, & setExecutable
```


File: Getting Info.

IO & NIO

```
// assuming the file exists & is empty  
File file1 = new File("./file1.txt");
```

```
long file1LastModified = file1.lastModified();  
// file1LastModified ==> 1759482505485  
// return value is ms since Jan 1, 1970
```

```
long file1Length = file1.length();  
// file1Length ==> 0
```

```
// see also setLastModified
```

File: Getting Info.

IO & NIO

```
// assuming the file exists & is not hidden  
File file1 = new File("./file1.txt");  
  
boolean file1Hidden = file1.isHidden();  
// file1Hidden ==> false
```

File: Manipulation

IO & NIO

```
// assuming the file does not exist but can be created  
File file1 = new File("./file1.txt");
```

```
boolean file1Created = file1.createNewFile();  
// file1Created ==> true
```

```
boolean file1Renamed = file1.renameTo(new File("file2.txt"));  
// file1Renamed ==> true  
// may also move the file!
```


File: Manipulation

IO & NIO

```
// assuming the file exists and can be deleted  
File file1 = new File("./file1.txt");
```

```
boolean file1Deleted = file1.delete();  
// file1Deleted ==> true
```

```
// assuming the directory does not exist but can be created  
File dir1 = new File("dir1");
```

```
boolean dir1Created = dir1.mkdir();  
// dir1Created ==> true
```

File: Manipulation

IO & NIO

```
// assuming the directories do not exist and can be created
```

```
File parentDir = new File("parent");
```

```
File childDir  = new File(parent, "child");
```

```
boolean directoriesCreated = childDir.mkdirs();
```

```
// directoriesCreated ==> true
```

```
// NB: calling mkdirs on the parent will not create the child
```


File

IO & NIO

- The returning of a boolean to indicate success/failure is bad practice
- No indication of why the operation failed
- Requires the caller to code an if statement
- One reason why `java.nio.file.Path` is preferred over `java.io.File`

Path: Creation

IO & NIO

// pre Java SE 8 before interface static methods
Path file1 = Paths.get("file1.txt");



// post Java SE 8

Path file1 = Path.of("file1.txt");



Path dir1 = Path.of("dir1");

Path file2 = Path.of("dir1", "file2.txt");

Path file4 = Path.of(new URI("file:///home/file4.txt"));

Path: Getting Elements

IO & NIO

```
Path file1 = Path.of("parent/child/file1.txt");
```

```
String file1AsString = file1.toString();  
// file1AsString ==> parent/child/file1.txt
```

```
Path file1Name = file1.getFileName();  
// file1Name ==> file1.txt
```

```
Path file1Segment2 = file1.getName(1);  
// file1Segment2 ==> child
```


Path: Getting Elements

IO & NIO

```
Path file1 = Path.of("parent/child/file1.txt");
```

```
int file1NumSegments = file1.getNameCount();  
// file1NumSegments ==> 3
```

```
Path file1last2Segments = file1.subpath(1, 3);  
// file1last2Segments ==> child/file1.txt
```

```
Path file1Parent = file1.getParent();  
// file1Parent ==> parent/child
```

Path: Getting Elements

IO & NIO

```
Path file1 = Path.of("parent/child/file1.txt");
```

```
Path file1Root = file1.getRoot();  
// file1Root ==> null
```

```
Path file1 = Path.of("/home/parent/child/file1.txt");
```

```
Path file1Root = file1.getRoot();  
// file1Root ==> /
```

Path: Removing Redundancies

IO & NIO

```
Path file1 = Path.of("parent/dir1/../child/file1.txt");
```

```
Path normalisedFile1 = file1.normalize();  
// normalisedFile1 ==> parent/child/file1.txt
```


Path: Converting

IO & NIO

```
Path file1 = Path.of("parent/child/file1.txt");
```

```
Path file1Uri = file1.toUri();  
// file1Uri ==> file:///home/parent/child/file1.txt
```

```
Path absoluteFile1 = file.toAbsolutePath();  
// absoluteFile1 ==> /home/parent/child/file1.txt
```

```
Path realFile1 = file.toRealPath();  
// realFile1 ==> /home/parent/child/file1.txt
```

Will resolve symbolic links and
throw if the file does not exist

Path: Joining

IO & NIO

```
Path dir1 = Path.of("parent/child");
```

```
Path file1 = dir1.resolve("file1.txt");  
// file1 ==> parent/child/file1.txt
```

```
Path file2 = dir1.resolve(Path.of("file2.txt"));  
// file2 ==> parent/child/file2.txt
```


Path: Creating a Path between Paths

IO & NIO

```
Path file1 = Path.of("parent/child/file1.txt");  
Path file2 = Path.of("parent/file2.txt");
```

```
Path fromFile1ToFile2 = file1.relativize(file2);  
// fromFile1ToFile2 ==> ../../file2.txt
```

```
Path fromFile2ToFile1 = file2.relativize(file1);  
// fromFile2ToFile1 ==> ../child/file1.txt
```

```
// NB: both Paths must be relative or absolute, not mixed
```

Path: Comparing IO & NIO

```
Path path = Path.of("parent/child");
```

```
boolean endsWithChild = path.endsWith("child");  
// endsWithChild ==> true
```

```
boolean startsWithParent = path.startsWith(Path.of("parent"));  
// startsWithParent = true
```

FileSystem

IO & NIO

```
FileSystem fs = FileSystems.getDefault();
```

```
// get drive letters (Windows)
```

```
Iterable<Path> paths = fs.getRootDirectories();
```

```
// get path segment separator
```

```
String separator = fs.getSeparator();
```

```
// avoid OS-specific code by creating Paths like this:
```

```
Path path = Path.of("parent", "child", "file1.txt");
```


Files

IO & NIO

- Path has no methods for getting info. or manipulating the FS
- For that we must use `java.nio.file.Files`
- `Files` has static methods, most of which accept a `Path/Paths`
- `Files` methods throw more exceptions than do `File` methods

Files: Getting Info.

IO & NIO

```
// assuming the file exists
```

```
Path file1 = Path.of("file1.txt");
```

```
boolean file1Exists = Files.exists(file1);
```

```
// file1Exists ==> true
```

```
boolean file1NotExists = Files.notExists(file1);
```

```
// file1NotExists ==> false
```

Files: Getting Info.

IO & NIO

```
// assuming a symbolic link; the linked file does not exist  
Path file1 = Path.of("file1");
```

```
boolean linkExists =  
    Files.exists(file1, LinkOption.NOFOLLOW_LINKS);  
// linkExists ==> true
```

```
boolean fileExists = Files.exists(file1);  
// fileExists ==> false
```


Files: Getting Info.

IO & NIO

```
// assuming the file exists & is readable & writable  
Path file1 = Path.of("file1.txt");
```

```
boolean file1Readable = Files.isReadable(file1);  
// file1Readable ==> true
```

```
boolean file1Writable = Files.isWritable(file1);  
// file1Writable ==> true
```

Files: Getting Info.

IO & NIO

```
// assuming the file exists but is not executable
```

```
Path file1 = Path.of("file1.txt");
```

```
boolean file1Executable = Files.isExecutable(file1);
```

```
// file1Executable ==> false
```


Files: Getting Info.

IO & NIO

```
// assuming the file exists & is empty
```

```
Path file1 = Path.of("file1.txt");
```

```
FileTime file1LastModified = Files.getLastModifiedTime(file1);
```

```
// file1LastModified ==> 2025-10-03T12:51:17.515530501Z
```

```
long file1Length = Files.size(file1);
```

```
// file1Length ==> 0
```

```
// see also setLastModifiedTime
```

Files: Getting Info.

IO & NIO

```
// assuming the file exists & is not hidden  
Path file1 = Path.of("file1.txt");
```

```
boolean file1Hidden = Files.isHidden(file1);  
// file1Hidden ==> false
```

Files: Manipulation

IO & NIO

```
Path file1 = Path.of("file1.txt");  
Path file2 = Path.of("file2.txt");
```

```
Path path = Files.createFile(file1);  
// path ==> file1.txt
```

```
Path path = Files.move(file1, file2);  
// path ==> file2.txt
```

No rename method;
use move to rename

Files: Manipulation

IO & NIO

```
// assuming the files exist and can be deleted
```

```
Path file1 = Path.of("file1.txt");
```

```
Path file2 = Path.of("file2.txt");
```

```
Files.delete(file1);
```

```
boolean file2Deleted = Files.deleteIfExists(file2);
```

```
// file2Deleted ==> true
```

Files: Manipulation

IO & NIO

```
// assuming the file and directory exist
```

```
Path path1 = Path.of("file1.txt");
```

```
Path path2 = Path.of("dir1", "file1.txt");
```

```
Path path = Files.copy(path1, path2);
```

```
// path ==> dir1/file1.txt
```

```
// the method may be passed one or more CopyOptions
```

```
import static java.nio.file.StandardCopyOption.*;
```

```
Path path = Files.copy(path1, path2, REPLACE_EXISTING);
```

Files: Manipulation

IO & NIO

```
// assuming the file and directory exist
```

```
Path path1 = Path.of("file1.txt");
```

```
Path path2 = Path.of("dir1", "file1.txt");
```

```
Path path = Files.move(path1, path2);
```

```
// path ==> dir1/file1.txt
```

```
// the method may be passed one or more CopyOptions
```

```
import static java.nio.file.StandardCopyOption.*;
```

```
Path path = Files.move(path1, path2, REPLACE_EXISTING);
```


Files: Manipulation

IO & NIO

```
// assuming the directory does not exist but can be created  
Path dir1 = Path.of("dir1");
```

```
Path path = Files.createDirectory(dir1);  
// path ==> dir1
```

```
// assuming the directories do not exist and can be created  
Path parent = Path.of("parent", "child");
```

```
Path path = Files.createDirectories(parent);  
// path ==> parent/child
```

Files: Listing Directory Content

IO & NIO

```
Path dir1 = Path.of("dir1");
```

```
DirectoryStream stream = Files.newDirectoryStream(dir1);
```

```
// DirectoryStream is an Iterable; not a java.util.Stream  
for (Path path : stream) {  
    System.out.println(path);  
}
```


Globbing (Pattern Matching)

IO & NIO

Special Character(s)	Description
*	Any number of characters
**	Like * but crosses directory boundaries
?	Exactly one character
{java, class}	Matches "java" or "class"
[abc]	Matches "a" or "b" or "c"
[0-9]	Matches any digit

Files: Listing Directory Content

IO & NIO

```
// assuming dir1 contains users.txt, App.java, & App.class  
Path dir1 = Path.of("dir1");
```

```
DirectoryStream stream =  
    Files.newDirectoryStream(dir1, "App.*");
```

Glob

```
for (Path path : stream) {  
    System.out.println(path);  
}
```

```
// output App.java\nApp.class
```

Files: Attributes

IO & NIO

```
// assuming file exists
```

```
Path file1 = Path.of("file1.txt");
```

```
boolean file1IsRegularFile = Files.isRegularFile(file1);
```

```
// file1IsRegularFile ==> true
```

```
boolean file1IsDirectory = Files.isDirectory(file1);
```

```
// file1IsDirectory ==> false
```

```
boolean file1IsLink = Files.isSymbolicLink(file1);
```

```
// file1IsLink ==> false
```


Files: Attributes

IO & NIO

```
// assuming UNIX; owner fred; rw-r----- (640) permissions
```

```
Path file1 = Path.of("file1.txt");
```

View: basic, dos, or posix

```
Object owner = Files.getAttribute(file1, "posix:owner");
```

```
// owner ==> fred
```

```
Object permissions = Files.getAttribute(  
    file1, "posix:permissions");
```

```
// permissions ==> [OWNER_READ, OWNER_WRITE, GROUP_READ]
```

```
// see also setAttribute and readAttributes
```

Walking a File Tree

IO & NIO

- To visit and optionally act upon each file and directory in a tree
- Searching for a file/directory is the classic use case
- Requires an implementation of `java.nio.file.FileVisitor`
- `SimpleFileVisitor` provides do-nothing method implementations (so you need only override the method(s) of interest)

Walking a File Tree

IO & NIO

```
class MyFileVisitor implements FileVisitor<Path> {  
  
    @Override  
    public FileVisitResult preVisitDirectory(  
        Path path, BasicFileAttributes attributes) throws IOException {  
        // TODO  
        return FileVisitResult.CONTINUE;  
    }  
  
    // TODO implement visitFile, visitFileFailed, & postVisitDirectory  
}
```

Other values: SKIP_SIBLINGS, SKIP_SUBTREE, TERMINATE

Walking a File Tree

IO & NIO

```
Path myHomeDir = Path.of("/home/fred");  
FileVisitor<Path> myVisitor = new MyFileVisitor();  
  
Files.walkFileTree(myHomeDir, myVisitor);  
  
// or...  
var options = Set.of(FileVisitOption.FOLLOW_LINKS);  
int maxDepth = 10;  
Files.walkFileTree(myHomeDir, options, maxDepth, myVisitor);
```

Watching a Directory for Changes

IO & NIO

1. Create a `WatchService` (watcher)
2. Register one or more directories (`Paths`) to watch
3. Poll the `WatchService`'s queue for `WatchKeys` in an infinite loop
 - a. Obtain `WatchEvents` from the `WatchKey`
 - b. Process the `WatchEvent`
 - c. Reset the `WatchKey` so it can be signalled again

Watching a Directory for Changes

IO & NIO

```
WatchService service = FileSystems.getDefault().newWatchService();
Path homeDir = Path.of("/home");
WatchKey homeDirKey = homeDir.register(service, ENTRY_CREATE);
```

```
...
```

```
while (true) {
```

```
    WatchKey key = service.take();
```

take() is blocking, poll() is not

```
    List<WatchEvent<?>> events = key.pollEvents();
```

```
    events.forEach(event -> {
```

WatchEvent's kind method returns the event kind, e.g. ENTRY_CREATE, and the context method returns the affected Watchable (Path in this case)

```
        // TODO process event
```

```
    });
```

```
    key.reset();
```

```
}
```

Reading & Writing Small (Text) Files

IO & NIO

```
Path textFile = Path.of("words.txt");
```

```
List<String> lines = Files.readAllLines(textFile);
```

```
Path textFileCopy = Path.of("copy-of-words.txt");
```

```
Files.write(textFileCopy, lines, StandardOpenOption.WRITE);
```

Varargs; others, e.g. APPEND,
TRUNCATE_EXISTING, CREATE_NEW,
CREATE, DELETE_ON_CLOSE

Reading & Writing Small (Binary) Files

IO & NIO

```
Path imageFile = Path.of("image.jpeg");
```

```
byte[] bytes = Files.readAllBytes(imageFile);
```

```
Path imageFileCopy = Path.of("copy-of-image.jpeg");
```

```
Files.write(imageFileCopy, bytes);
```


Character Sets

IO & NIO

- A named mapping between characters and bytes
- Text files are encoded using a character set
- UTF8 is common (& the Java default since v18) but is not universal
- Attempts to read an otherwise encoded file without specifying the appropriate character set will result in an error
- Files methods that read & write files are overloaded to accept a `java.nio.charset.Charset`

Character Sets

IO & NIO

```
// assuming a file encoded using the GBK Charset  
Path file1 = Path.of("file1.txt");
```

```
if (Charset.isSupported("GBK")) {  
    Charset charset = Charset.forName("GBK");  
    List<String> lines = Files.readAllLines(path, charset);  
    ...  
}
```

```
// see also availableCharsets
```

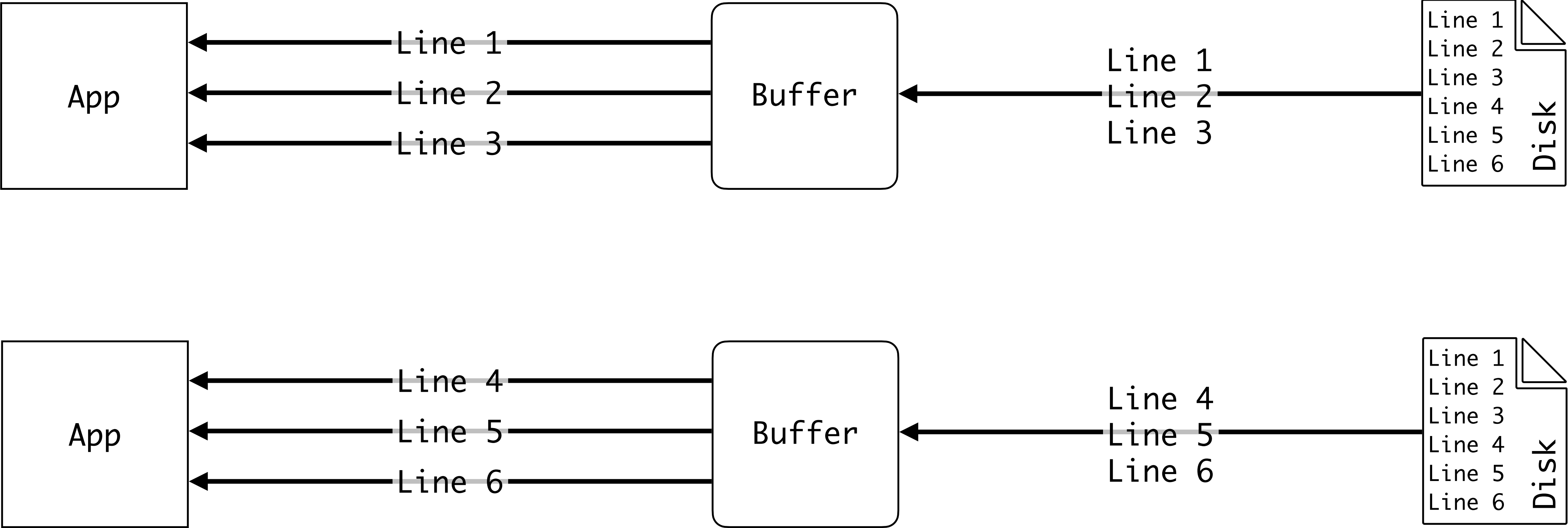
Buffering

IO & NIO

- Reading the whole of a large file at once may overwhelm memory
- Reading a large file line-by-line is slow (too many round trips to the disk)
- A buffer is a segment of memory into which some of the file is read
- Data is consumed from the buffer
- When the buffer is empty some more of the file is read into it

Buffering

IO & NIO



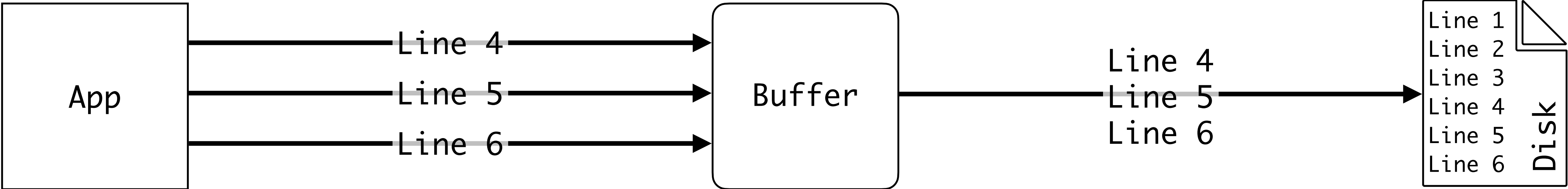
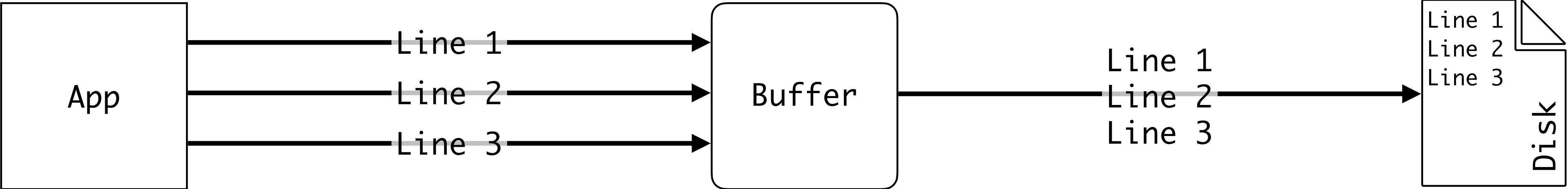
Buffering

IO & NIO

- Writing many lines to a file one-at-a-time is slow (too many round trips to disk)
- A buffer is a segment of memory into which some of the line are written
- When the buffer is full its contents are flushed (written) to disk
- The buffer is also flushed when the output stream is closed

Buffering

IO & NIO



Reading Text Files

IO & NIO

```
Path textFile = Path.of("words.txt");  
List<String> lines = new LinkedList<>();
```

```
BufferedReader reader = Files.newBufferedReader(textFile);  
String line = reader.readLine();  
while (line != null) {  
    Lines.add(line);  
    line = reader.readLine();  
}  
reader.close();
```

Instantiation typically
in a try-with-resources
block; BufferedReader
implements AutoCloseable

Writing Text Files

IO & NIO

```
Path textFileCopy = Path.of("copy-of-words.txt");
```

```
BufferedWriter writer = Files.newBufferedWriter(textFileCopy);  
    for (String line : lines) {  
        writer.write(line + "\n");  
    }  
}  
writer.close();
```

Overloaded to accept
Charset and OpenOptions

Reading Binary Files

IO & NIO

```
Path imageFile = Path.of("image.jpeg");  
int fileSize = (int) Files.size(imageFile);  
byte[] bytes = new byte[fileSize];
```

Should check the
file size is not >
Integer.MAX_VALUE

```
BufferedInputStream bin = new BufferedInputStream(  
    Files.newInputStream(imageFile));
```

```
bin.read(bytes);  
bin.close();
```

The read method
(no args) returns
the next byte or -1

Writing Binary Files

IO & NIO

```
Path imageFileCopy = Path.of("copy-of-image.jpeg");
```

```
BufferedOutputStream bos = new BufferedOutputStream(  
    Files.newOutputStream(imageFileCopy));
```

```
bos.write(bytes);  
bos.close();
```

Overloaded to accept
OpenOptions

Decoration

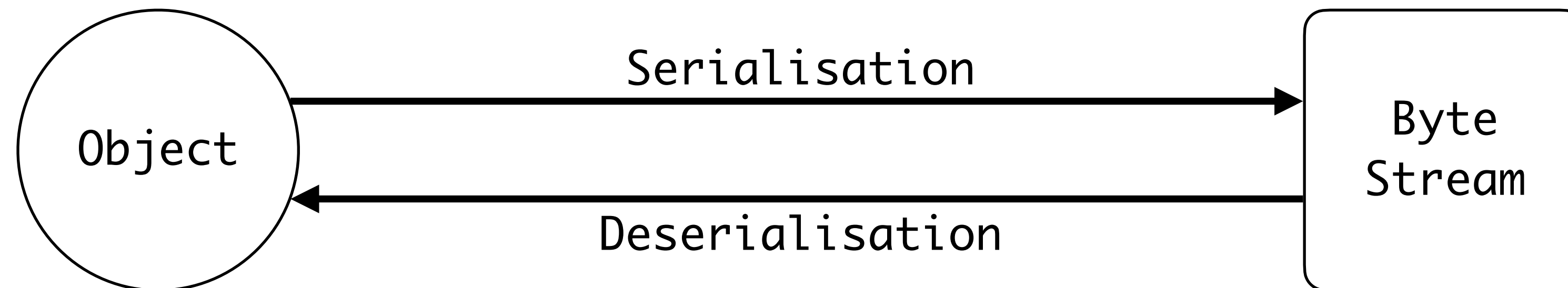
IO & NIO

- Classes in the java.io package exploit the decorator design pattern
- An object is composed of another and builds upon its behaviour, e.g.:
- BufferedOutputStream decorates OutputStream by buffering
- InputStreamReader decorates InputStream by reading characters
- DataOutputStream decorates OutputStream by writing primitives
- GZipInputStream decorates InputStream by reading compressed files

Serialisation

IO & NIO

- The conversion of an object to a byte stream
- To write objects to disk; to transfer objects over a network



Serializable

IO & NIO

- An interface in the `java.io` package
- Has no methods; signals the VM that instances may be serialised

```
class User implements Serializable {
```

```
    static final long serialVersionUID = 1L;
```

```
    String username;  
    String password;
```

```
}
```

The process is recursive - all fields MUST be Serializable; static and transient fields are not included

Used to verify compatible classes when deserialising

ObjectOutputStream

IO & NIO

```
Path userFile = Path.of("user.obj");  
User user = new User("fred", "Fr3ddy");
```

```
ObjectOutputStream oos = new ObjectOutputStream(  
    Files.newOutputStream(userFile));
```

```
oos.writeObject(user);  
oos.close();
```

ObjectInputStream

IO & NIO

```
Path userFile = Path.of("user.obj");
```

```
ObjectInputStream ois = new ObjectInputStream(  
    Files.newInputStream(userFile));
```

```
User user = (User) ois.readObject();  
ois.close();
```